FDS COURSE PROJECT · PGDBA BATCH 12 · IIM CALCUTTA

PlaceIQ

A campus placement & recruitment database that enforces its own rules

Fundamentals of Database Systems · Prof. Debapriyay Mukhopadhyay

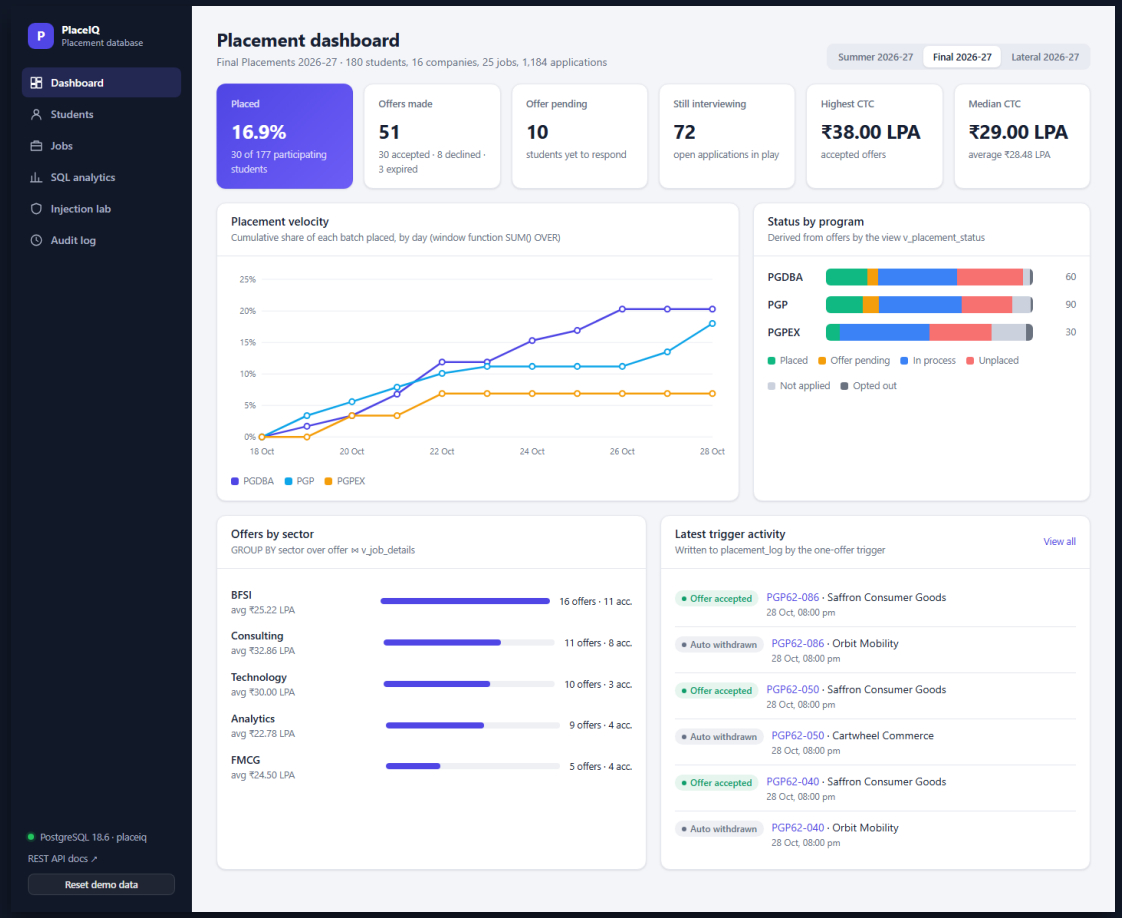
M1 Ekta Sharma

M2 [Member 2]

M3 [Member 3]

M4 [Member 4]

November 2026



Live web UI over PostgreSQL 18: every number on screen is computed by SQL

Four parts, four presenters

M1 Ekta Sharma

M1

Ekta Sharma

Problem & EER

0:00 – 3:45

- Problem & anomalies
- Business rules
- EER model: specialization, weak entity, aggregation, union type

M2

[Member 2]

Schema & normalization

3:45 – 7:30

- EER → relational mapping
- Keys: super, candidate, alternate, foreign
- FDs, 1NF → BCNF, lossless & dependency-preserving

M3

[Member 3]

DDL, data & queries

7:30 – 11:15

- DDL: constraints as rules
- Seeded, reproducible data
- Five advanced queries: division, windows, HAVING

M4

[Member 4]

Rules, transactions & performance

11:15 – 15:00

- Triggers & stored procedures
- Live demo, ACID & concurrency
- Indexing, SQL injection, testing, learnings

One spreadsheet, three anomalies

M1 Ekta Sharma

roll_no	company	hq_city	role	pay	round	result
DBA12-007	Northbridge	Mumbai	Assoc. Consultant	34 LPA	1 CASE	CLEARED
DBA12-007	Northbridge	Mumbai	Assoc. Consultant	34 LPA	2 CASE	SCHEDULED
DBA12-019	Northbridge	Mumbai	Assoc. Consultant	34 LPA	—	REJECTED
PGP62-003	Northbridge	Mumbai	Assoc. Consultant	34 LPA	1 CASE	CLEARED

Company, city, role and pay repeat on every applicant × round row

Update anomaly

Northbridge moves HQ: edit every row, miss one and the data contradicts itself.

Insertion anomaly

A new job cannot be recorded until a student applies (roll_no is part of the key).

Deletion anomaly

Delete the only applicant and the job's pay, cut-off and rounds disappear too.

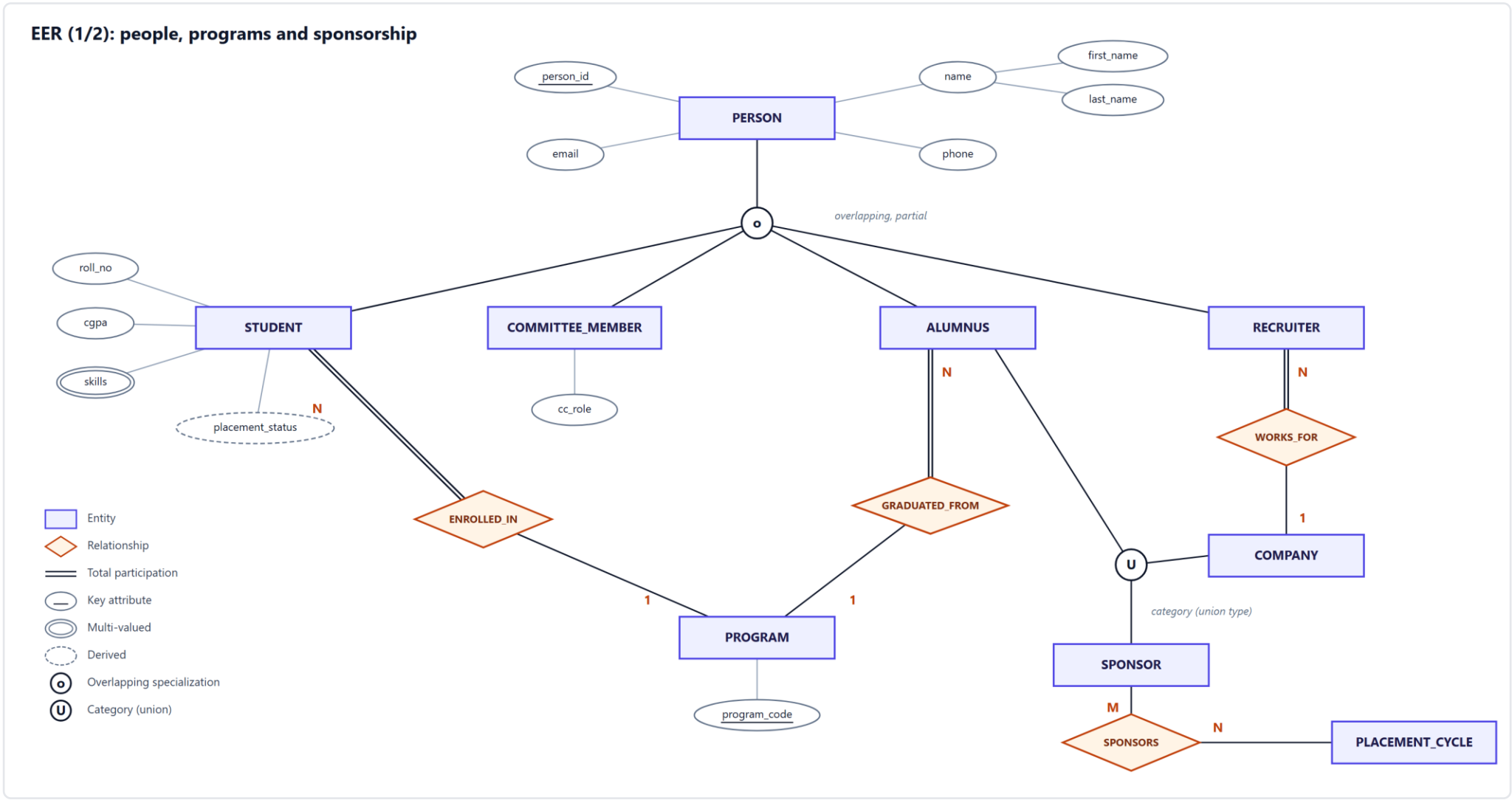
15

business rules, each enforced by the database, not by people

- Eligibility: program, CGPA cut-off, backlogs, deadline
- No overlapping interview slots
- One accepted offer per cycle, with automatic withdrawal elsewhere
- Placed status derived, never typed in
- A student holding an offer cannot be deleted

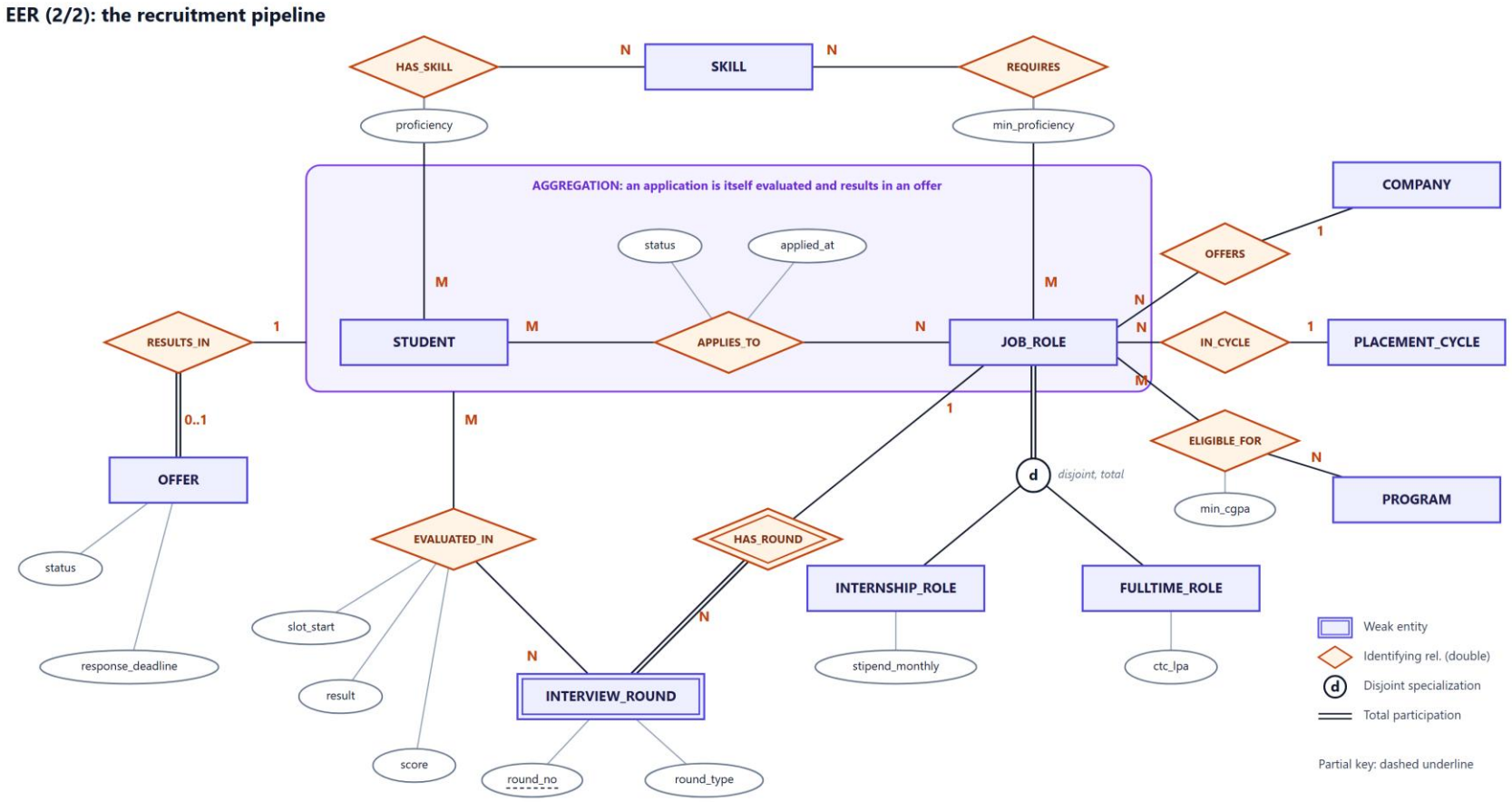
EER model (1/2): people, programs and sponsors

M1 Ekta Sharma



EER model (2/2): the recruitment pipeline

M1 Ekta Sharma



Aggregation

APPLIES_TO is treated as an entity: interviews and offers attach to an application, not to a student.

Weak entity

INTERVIEW_ROUND is identified by (job_id, round_no) through HAS_ROUND.

Disjoint & total

Every job is exactly one of internship or full-time ('d', double line).

From EER to 22 relations, and their keys

M2 [Member 2]

Step	Mapping rule	Applied to
1	Strong entity → table	program, person, company, job_role …
2	Weak entity → PK = owner PK + partial key	interview_round (job_id, round_no)
3–4	1:1 / 1:N → foreign key on the N side	student.program_id, offer → application
5	M:N → bridge table	application, eligibility, interview
6	Multi-valued attribute → own table	student_skill
8A	Specialization → table per subclass	student, recruiter; internship_role …
9	Category → surrogate key + FK in members	sponsor ← company, alumnus

Super key

Any attribute set that identifies a row: (person_id, email)

Candidate keys

Minimal super keys. placement_cycle has three: cycle_id, cycle_name, (cycle_type, academic_year)

Primary & alternate

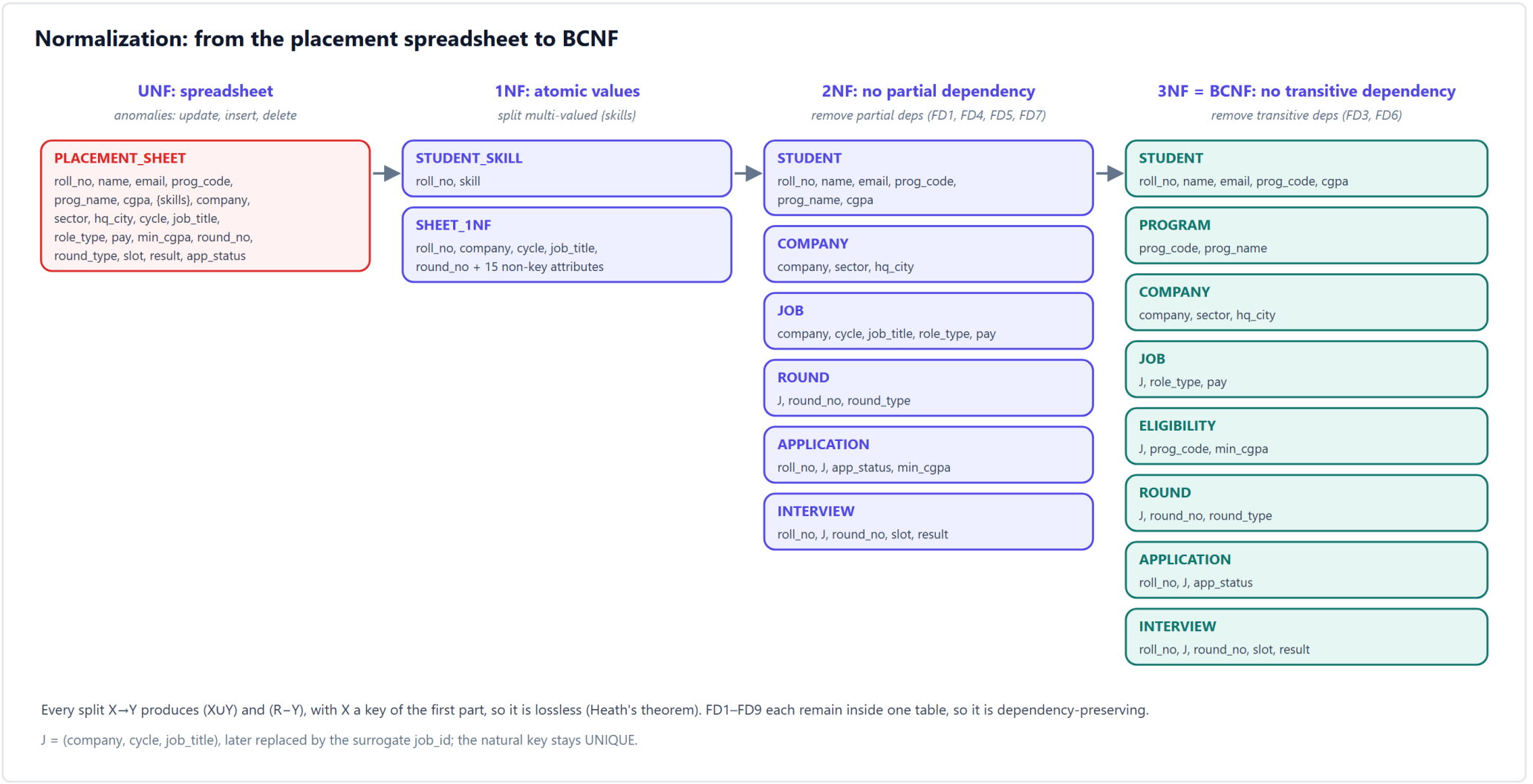
job_id is primary; (company_id, cycle_id, title) stays UNIQUE as the alternate key

Foreign keys

30 FKs, some composite: (posted_by, company_id) → recruiter

Normalization: spreadsheet → BCNF

M2 [Member 2]



Every split X→Y produces (XUY) and (R−Y), with X a key of the first part, so it is lossless (Heath's theorem). FD1–FD9 each remain inside one table, so it is dependency-preserving.

J = (company, cycle, job_title), later replaced by the surrogate job_id; the natural key stays UNIQUE.

Proving BCNF, and the trap we avoided

M2 [Member 2]

Relation	Non-trivial FDs	Determinants are keys?
student	roll_no → all; email → roll_no	Yes: both are candidate keys
program	prog_code → prog_name	Yes
eligibility	(job, program) → min_cgpa	Yes
interview_round	(job, round_no) → round_type	Yes
interview	(student, job, round) → slot, result	Yes
placement_cycle	3 candidate keys, each → all	Yes

All 22 relations: every determinant is a super key → BCNF

Deliberately not stored: CTC on the offer (offer → job → ctc would be transitive), placement status (derived by a view), cycle on the application (job → cycle).

The 3NF-but-not-BCNF trap

Suppose: each recruiter interviews for exactly one job.

Panel(student, job, recruiter) with (student, job) → recruiter and recruiter → job.

recruiter is not a super key → not BCNF. job is prime → still 3NF.

BCNF split: (recruiter, job) + (student, recruiter) is lossless but NOT dependency-preserving.

Our decision: the business does not need that rule, so we did not model it. The schema stays BCNF with nothing lost.

Integrity lives in the schema

M3 [Member 3]

```
-- Disjoint specialization, enforced declaratively
CREATE TABLE internship_role (
  job_id     INT PRIMARY KEY,
  role_type  VARCHAR(10) DEFAULT 'INTERNSHIP'
             CHECK (role_type = 'INTERNSHIP'),
  stipend_monthly NUMERIC(10,2) NOT NULL,
  FOREIGN KEY (job_id, role_type)
    REFERENCES job_role (job_id, role_type)
    ON DELETE CASCADE
);

-- No student in two interviews at once
CONSTRAINT excl_student_double_booking
  EXCLUDE USING gist (
    student_id WITH =,
    tstzrange(slot_start, slot_end) WITH &&)
```

45
CHECK constraints (domains)

30
foreign keys, with actions

17
UNIQUE (alternate keys)

1
EXCLUDE (overlap ban)

3
deferred constraint triggers

55
indexes incl. FK indexes

ON DELETE, chosen per relationship

CASCADE: subtypes & weak entity · RESTRICT: history (offers) · SET NULL (col): keep job, drop recruiter link

A realistic, reproducible test bed

M3 [Member 3]

180
students in 3 programs

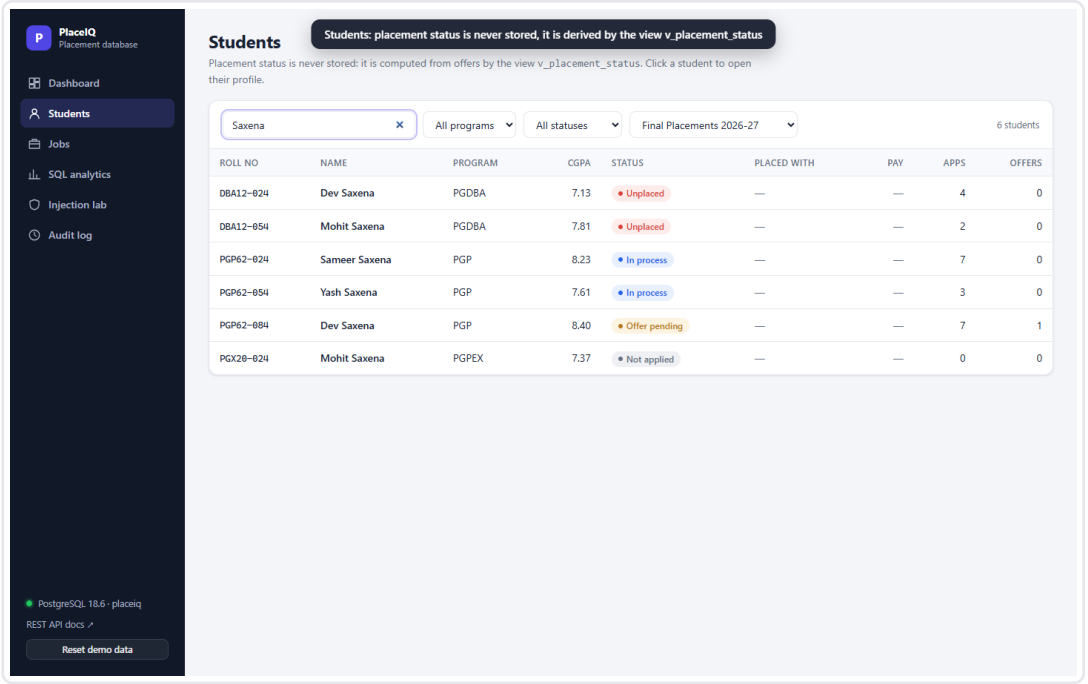
16
fictional companies

1,184
applications

1,057
interview slots

96
offers made

- Reference data is hand-written INSERTs, readable on a slide
- Students are generated set-based; setseed() makes every run identical
- A PL/pgSQL simulation runs shortlist → rounds → offers → accept, job by job
- Every row is loaded THROUGH the triggers, in one transaction: it all passed every rule, or nothing loads



Five queries, five business questions

M3 [Member 3]

#	Business question	SQL technique	Answer from our data
Q1	Which recruiters are hardest to crack?	Joins, outer join, GROUP BY, HAVING vs scalar subquery	5 companies; Meridian Insurance offers to 8.6%
Q2	Who holds EVERY skill a new job needs?	Relational division: double NOT EXISTS	8 students, led by DBA12-039
Q3	Top earners per program?	DENSE_RANK, NTILE, AVG() OVER on a view	Top CTC ₹38 LPA
Q4	How fast is each batch getting placed?	CTEs, generate_series, running SUM() OVER	20.3% of PGDBA placed by 28 Oct
Q5	Strong students lost in interviews?	Correlated subquery + LATERAL join	3 students flagged for coaching

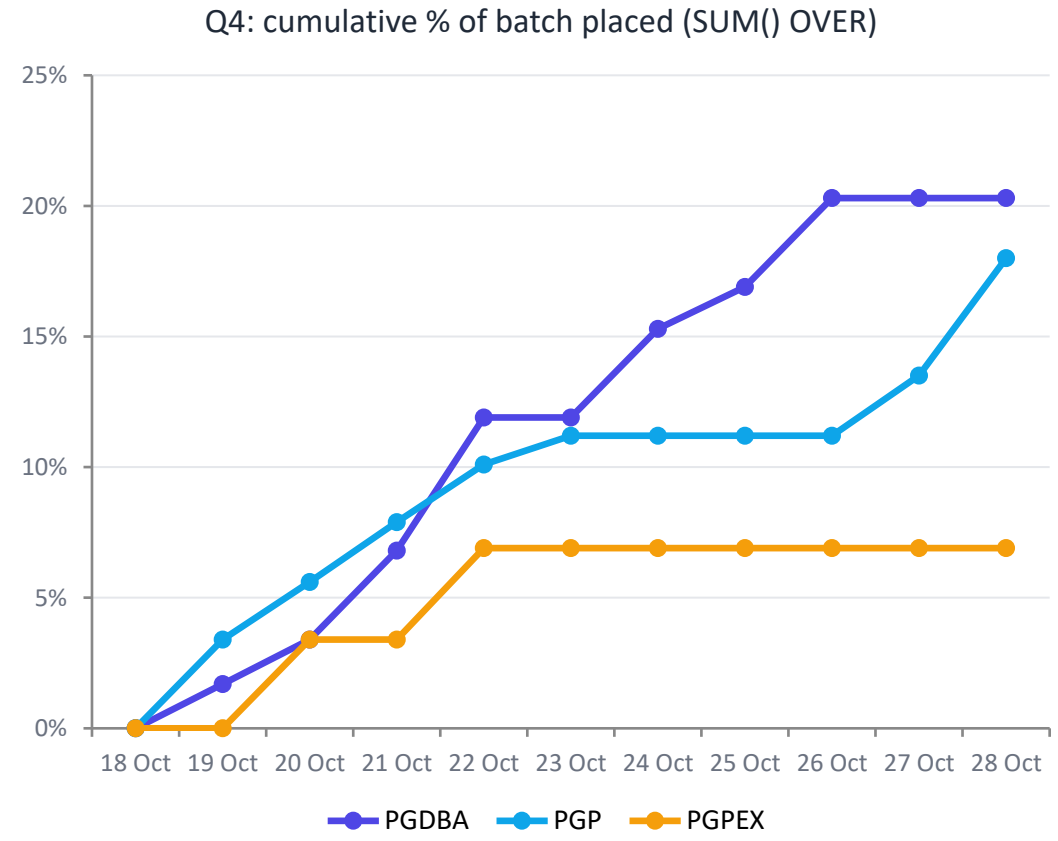
Spotlight: relational division and window functions

M3 [Member 3]

```
-- students holding EVERY skill job 25 needs
SELECT s.roll_no FROM student s
WHERE NOT EXISTS (          -- no required skill
  SELECT 1 FROM job_required_skill r
  WHERE r.job_id = 25
    AND NOT EXISTS (        -- ... that s lacks
      SELECT 1 FROM student_skill ss
      WHERE ss.student_id = s.person_id
        AND ss.skill_id   = r.skill_id
        AND ss.proficiency >= r.min_proficiency));
```

Qualified(student, skill) ÷ π skill (σ job = 25 (JobRequiredSkill))

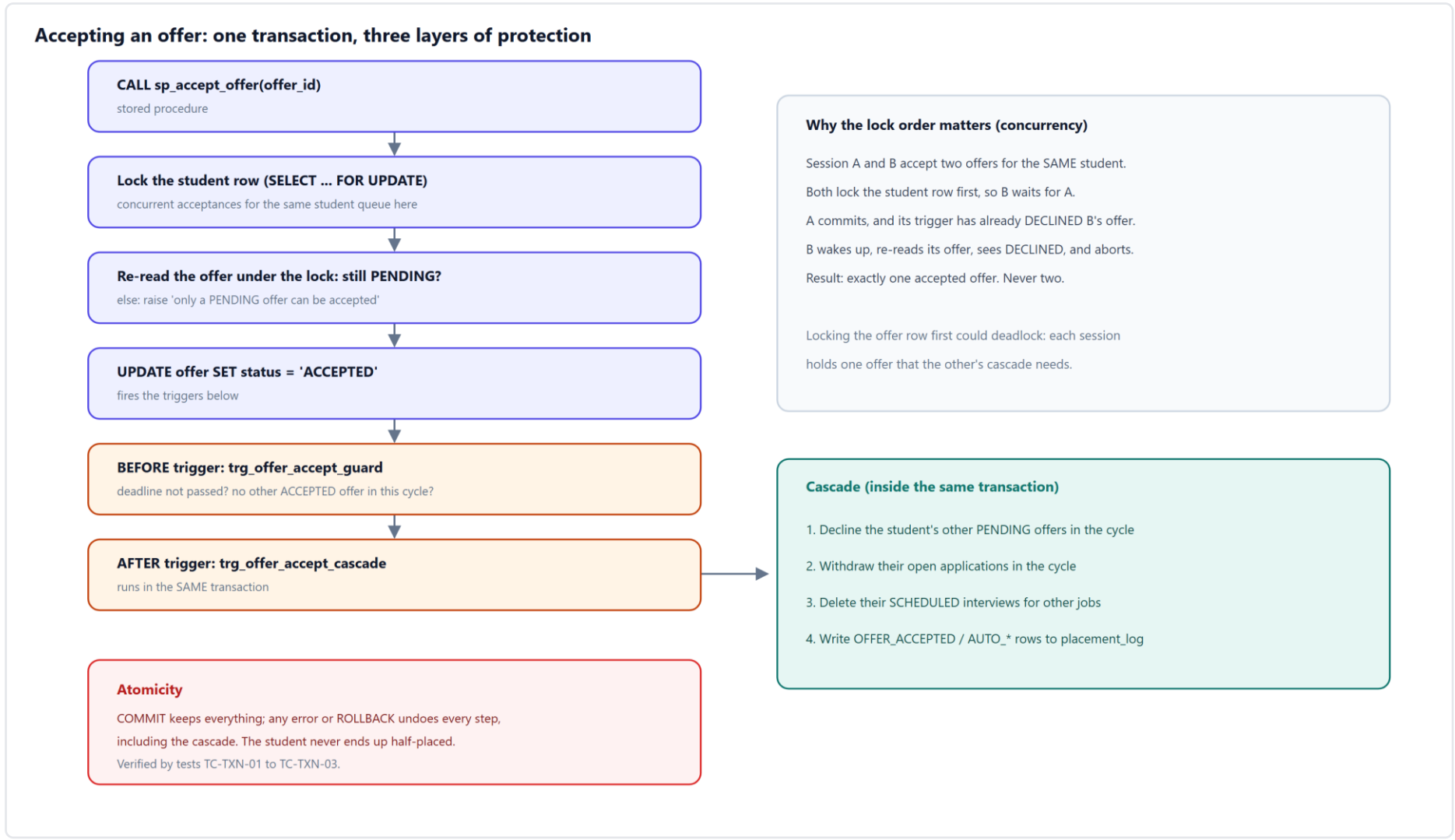
Cross-checked against division-by-counting (GROUP BY … HAVING count = …) for all 25 jobs: identical results (test TC-QRY-06).



Window functions aggregate without collapsing rows: each day keeps its own row and gets a running total.

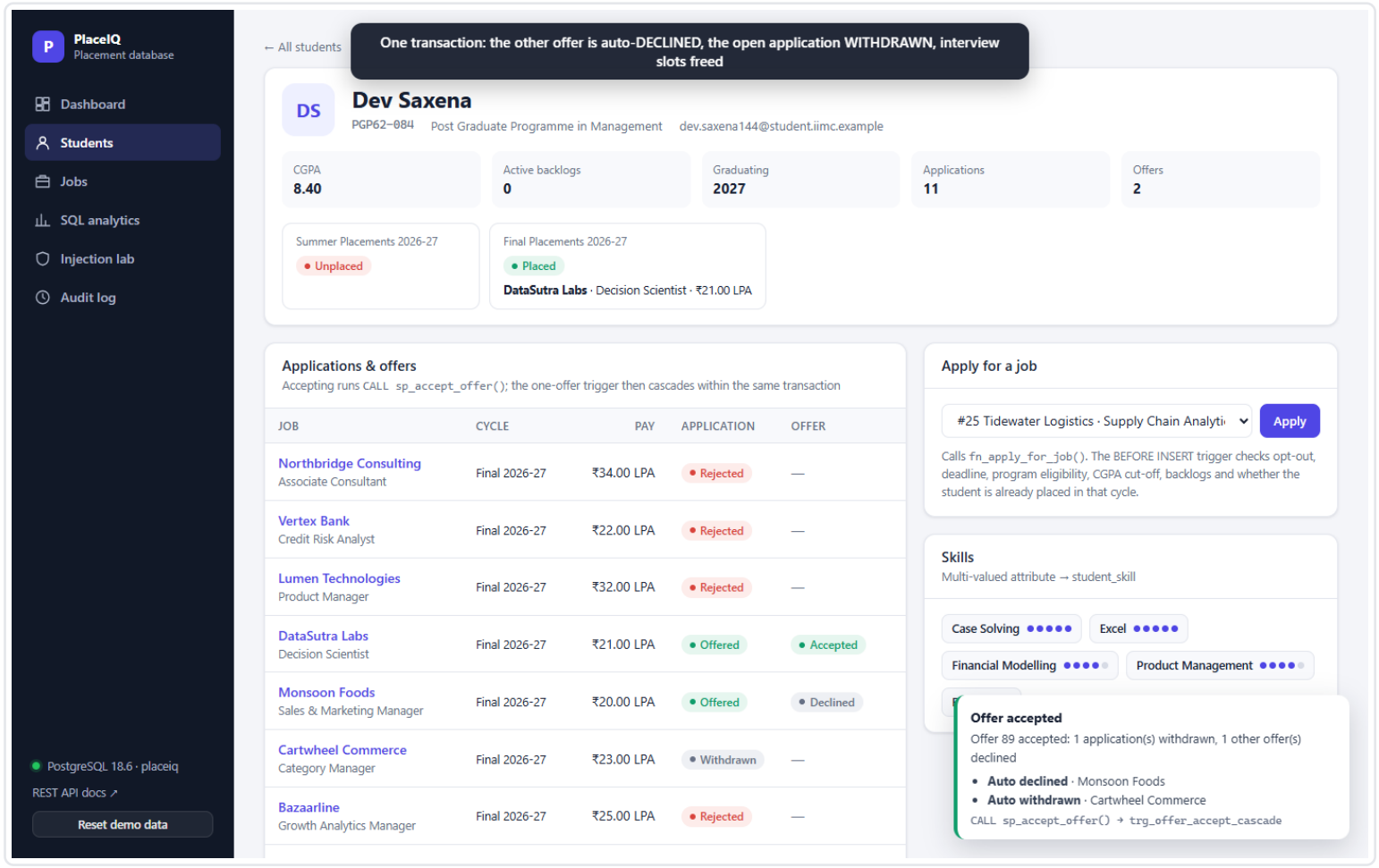
The one-offer policy, as a trigger

M4 [Member 4]



Live demo: accept an offer, watch the cascade

M4 [Member 4]



1. **Second offer**
   Monsoon Foods makes Dev Saxena an offer; trigger marks the application OFFERED

2. **Accept**
   Dev accepts DataSutra: CALL sp_accept_offer()

3. **Cascade**
   Other offer DECLINED, open application WITHDRAWN, interviews freed, all logged

4. **Guard**
   Applying again, or a CGPA below the cut-off, is refused with the trigger's message

Backup if the live demo fails: PlaceIQ_Demo_Video.webm

Transactions: ACID under concurrency

M4 [Member 4]

A Atomicity

ROLLBACK undoes the whole cascade: offers, applications, interviews, log (TC-TXN-01)

C Consistency

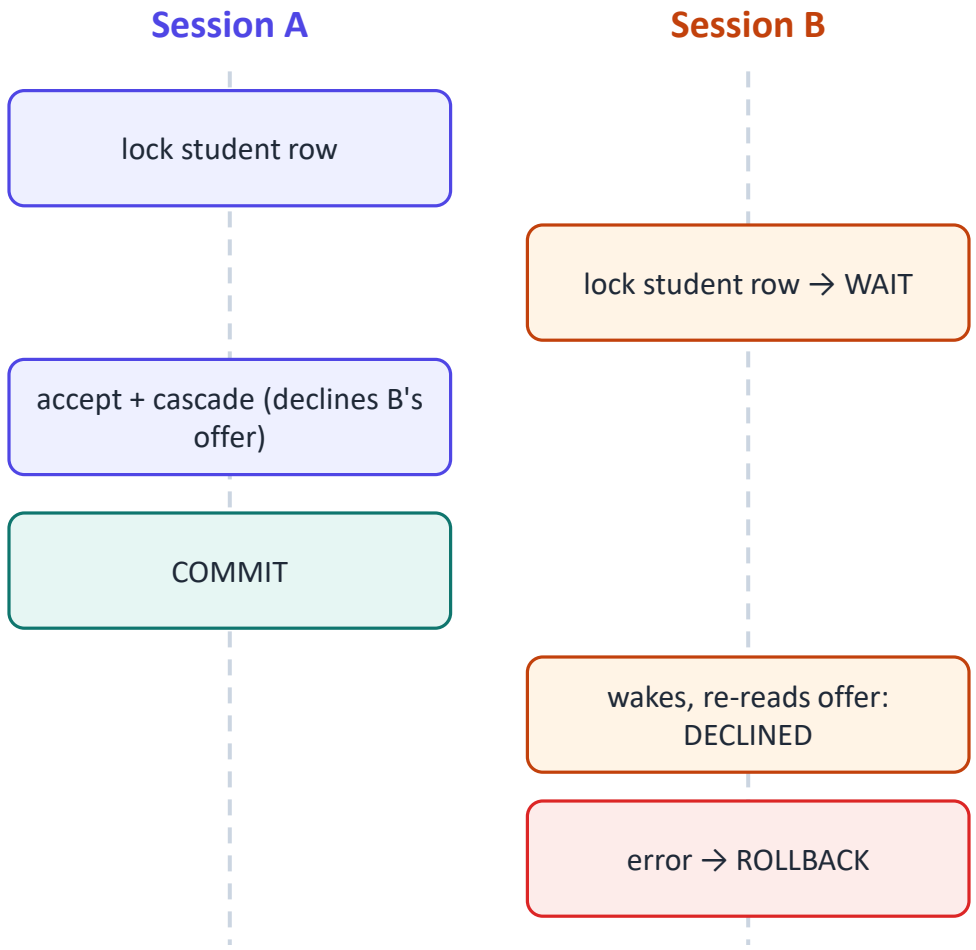
Constraints + deferred totality check at COMMIT; a job without a subtype cannot commit (TC-TXN-04)

I Isolation

Row lock on the student serialises concurrent acceptances (TC-TXN-02/03)

D Durability

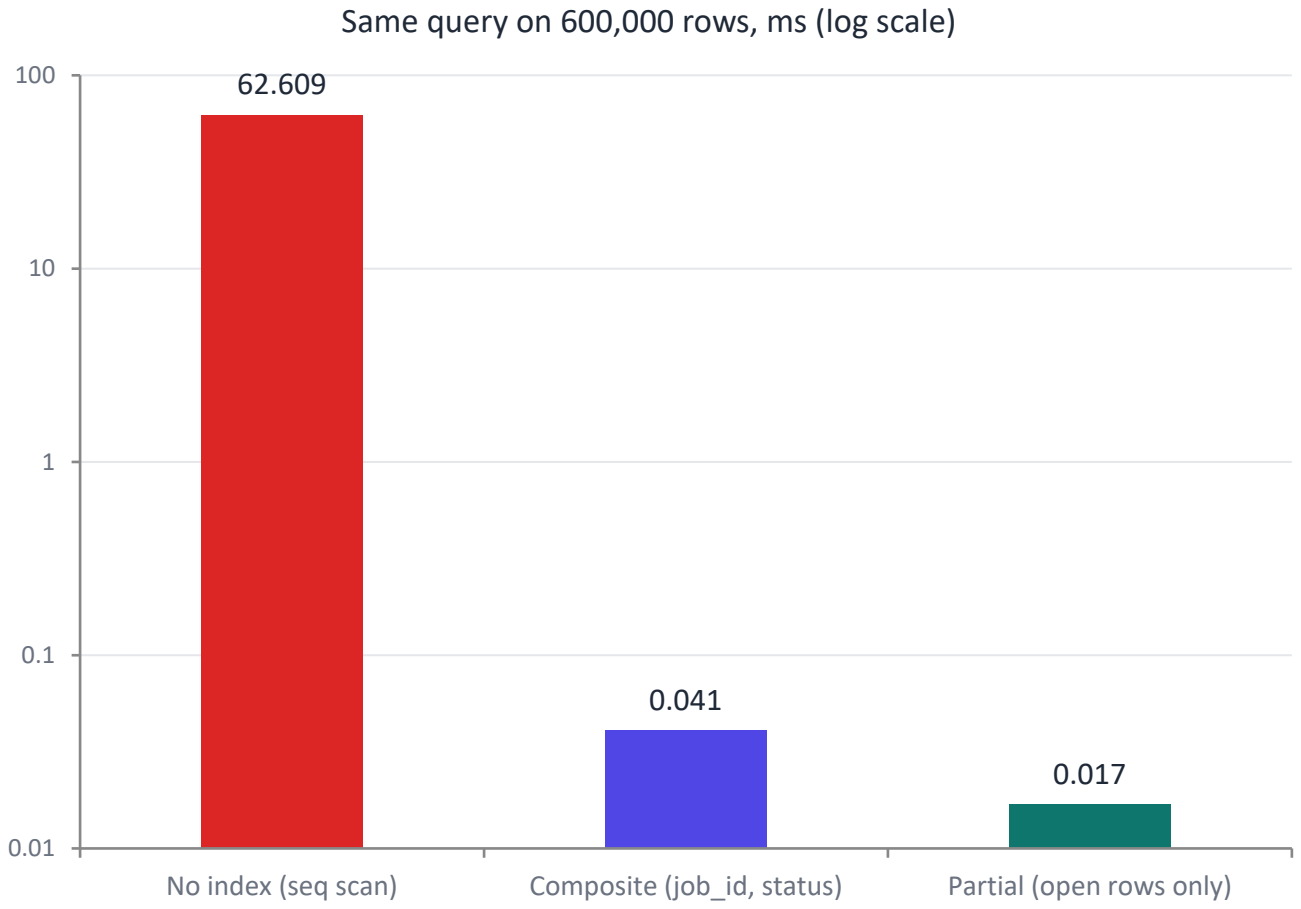
Once COMMIT returns, PostgreSQL's write-ahead log keeps it through a crash



Result: exactly one accepted offer. Locking the student first (not the offer) also rules out a deadlock.

Indexing: measured, not assumed

M4 [Member 4]



1,527× faster

composite B-tree index vs sequential scan

26× smaller

partial index (176 KB) vs full index (4,488 KB)

3 vs 4,383 blocks

leading-column rule: filter on status alone cannot seek the (job_id, status) index

Measured with EXPLAIN (ANALYZE, BUFFERS) by tests TC-PRF-01 to 03.

Security and testing

M4 [Member 4]

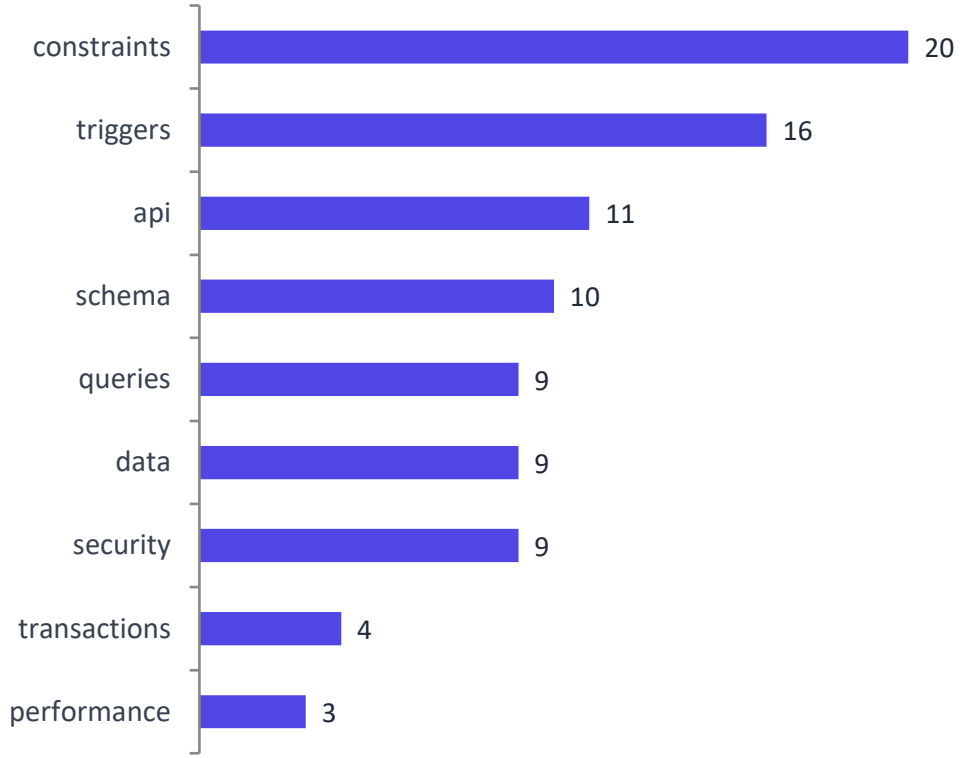
SQL injection, same input x' OR '1'='1

180
rows leaked: string concatenation

0
rows: bound parameter ($1)

- Defence in depth: the unsafe path runs as role placeiq_readonly (GRANT SELECT on 2 tables) in a READ ONLY transaction
- DROP TABLE payload → SQLSTATE 25006; pg_read_file → 42501 (permission denied)
- Every other query in the app uses parameters

91 / 91
automated tests passed (pytest, real database)



What we learned

M1 Ekta Sharma

Modelling decides everything downstream: treating the application as an aggregation is what made interview and offer integrity enforceable.

M2 [Member 2]

Normalization is a proof: writing every FD exposed the transitive dependency hiding in min_cgpa.

M3 [Member 3]

Declare before you code: CHECK, composite FKs and EXCLUDE reject bad data even if the app is bypassed.

M4 [Member 4]

Concurrency bugs stay invisible until tested: lock order turned a possible deadlock into a clean queue.

Thank you. Questions?

github.com/EktaSharma-github/placeiq · schema, data, queries, tests and the web UI